SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	A. DEKKSHITHA	**Reg. No.:**	192572133
Date:	10.10.2026	**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification	Selects the most suitable SDLC/model/frame	Suitable methodology selected with	Methodology selected but justification is	Outcomes, limitations, or future

(10%)	work and provides strong technical justification.	adequate justification.	weak.	enhancements are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

Smart Campus Lost-and-Found Management System

1. Understand the Problem Statement

1.1 Problem Explanation

Students, faculty members and campus staff frequently lose belongings such as ID cards, wallets, phones, electronic devices, books, bags and accessories. When an item is lost, its owner may not know where to report it; a person who finds an item may not know how to identify its owner. Current methods rely on notice boards, verbal communication, group messages and manual registers. Information can be scattered, outdated or incomplete, making matching and recovery slow. The proposed Smart Campus Lost-and-Found Management System is a centralized software application for reporting lost and found items with descriptions, photographs, locations, dates and timestamps. It identifies potential matches, notifies users, records recovery history and supports secure handover after ownership verification.

1.2 Objectives

- Register lost and found items with descriptions and photographs. Record the location, date and time of loss or discovery.
- Search reports and identify potential lost–found matches. Notify users about possible matches and support claim requests.
- Verify ownership before handing over recovered belongings. Maintain item status, recovery history and administrative records. Generate analytics on item categories, locations and recovery success.

1.3 Scope

The system is intended for students, faculty, campus staff and authorized administrators within a college or university. It covers item reporting, photograph uploads, searching and filtering, potential-match identification, notifications, claim verification, handover records, recovery history and analytics. The first version focuses on belongings lost within the campus. Multi-campus support and advanced image recognition may be added later.

2. Key Stakeholders and Challenges

2.1 Stakeholders, Roles and Responsibilities

- Students: report lost belongings, register found items, search reports, receive alerts and claim their property.
- Faculty members: report items, help students identify belongings and guide the recovery process.
- Campus staff/security: register items found in classrooms, laboratories, libraries and common areas; safeguard items and assist with handover.

- System administrator: manage accounts and permissions, review reports, remove duplicate or inappropriate entries and maintain records.
- College management: establish policies, provide resources and review recovery analytics.
- Item owner/claimant: provide suitable evidence of ownership and collect the item after verification.

2.2 Major Challenges

- Scattered information makes it difficult to know where an item was found.
- Manual searching and contacting multiple people takes time.
- Incomplete descriptions, photographs, dates or locations make identification harder.
- Personal details and photographs require privacy protection.

3. Analysis of the Current Approach

3.1 Existing Process

When someone loses an item, they may inform a teacher, contact campus security, check a notice board or post in a class group. A finder may submit the item to an office or inform others informally. Staff may write details in a register. The owner then has to check different sources or contact several people to find the item.

3.2 Strengths

- Simple and familiar for users; little technical knowledge is needed.
- Low initial cost and minimal infrastructure requirements.
- Direct communication allows people to ask questions.
- Campus staff can physically safeguard and return recovered items.

3.3 Limitations and Gaps

- No central database: information is spread across registers, notices and messages.
- Slow search: owners must manually check multiple sources.
- Limited status tracking: reports may not show whether an item is pending, claimed or returned.
- Paper records can be lost, while duplicate or outdated entries may remain.

4. Proposed Software Solution and Software Engineering Principles

4.1 Proposed Solution

The system will be a web-based application with a centralized database. Users log in, report lost or found items and provide the item name, category, description, photograph (if available), location, date and time. The application stores reports and compares relevant details to suggest potential matches.

Users are notified of relevant matches and can submit claims. Authorized staff verify ownership before recording handover and updating the item status.

4.2 Major Functional Modules

Module 1: Lost and Found Item Registration

Register lost and found items and enter descriptions, category, location, date and time. Upload photographs where available; edit or update reports. Validate required fields and store reports in the database.

Module 2: Item Search and Potential-Match Identification

Search by keywords and filter by category, location and date. Compare lost and found reports using descriptions and available attributes. Display possible matches for users to review and confirm.

Module 3: Notifications and Secure Item Recovery

Notify users about potential matches. Accept claim requests and support ownership verification by authorized staff. Record handover, update item status and maintain recovery history.

Module 4: Administration and Analytics

Manage user accounts, permissions and reports. Track pending, matched, claimed and returned items. Generate statistics on recovery rates, item categories and frequently affected locations.

4.3 Functional Requirements

- The system shall support secure login and role-appropriate access.
- It shall register, store, edit and retrieve lost and found reports.
- It shall capture descriptions, categories, photographs, locations, dates and timestamps.
- It shall provide search, filtering and potential-match identification.
- It shall notify users of potential matches and allow claim requests.

4.4 Non-Functional Requirements

- Usability: clear forms, simple navigation and understandable status labels.
- Reliability: accurate storage, error handling and regular backups.
- Security: authentication, role-based permissions and protection of personal information.
- Performance: searches and comparisons should respond within a reasonable time.
- Scalability: support growth in users, reports and photographs.

4.5 Software Engineering Principles

- Modularity: four separate modules have clear responsibilities and can be developed and tested individually.

- Scalability: database and application design should accommodate more users and records without a full redesign.
- Usability: straightforward forms, filters and status labels make reporting and recovery easier.
- Security: authentication, role-based access and restricted personal information reduce misuse; ownership checks reduce fraudulent claims.
- Reusability and testability: shared validation and permission functions can be reused, and each module can be tested before integration.

5. Chosen Methodology / Framework

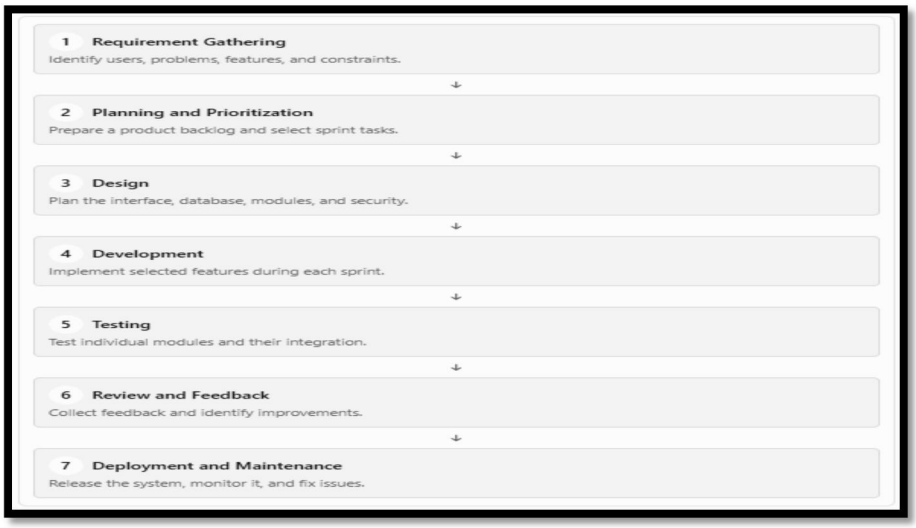
5.1 Selected Methodology: Agile

Agile is an iterative and incremental software development approach. The project is divided into smaller development cycles (sprints), with each cycle delivering features that can be tested and improved using feedback. Agile is suitable because the application has several connected modules and requirements may change after students, faculty and staff try early versions.

5.2 Justification

- Incremental delivery: registration can be built first, followed by matching, recovery and analytics.
- Continuous feedback: campus users can suggest improvements to forms and workflows.
- Early testing: defects in searching, matching, notifications and status updates can be found sooner.
- Flexibility: new or revised requirements can be added to the backlog.
- Risk reduction: technical and usability problems can be addressed before final release.

5.3 Agile Process for This Project



For example, an early sprint may deliver login and item reporting; the next may deliver searching and matching; later sprints can add notifications, secure recovery and analytics. Testing and feedback repeat during development. Agile therefore supports progressive delivery, changing requirements, improved quality and ongoing maintenance.

6. Expected Outcomes, Limitations and Future Enhancements

6.1 Expected Benefits

- Faster recovery through centralized records, searching and match alerts.
- Better organization of reports than scattered messages and paper registers.
- Reduced administrative work through automated searching and status tracking.
- Greater participation by students, faculty and staff through a shared platform.
- Analytics to identify common item categories, locations and recovery trends.

6.2 How the Solution Addresses Challenges

Centralized storage addresses scattered records; search and filters reduce manual searching; potential-match identification assists comparison; notifications reduce missed communications; structured forms improve report completeness; verification helps prevent fraudulent claims; status tracking preserves recovery information; and analytics provide campus-wide statistics.

6.3 Risks, Challenges and Limitations

- Incorrect matches may occur when different items have similar descriptions; users must review and confirm them. Incomplete or inaccurate reports can reduce matching effectiveness.
- Personal details and photographs create privacy risks, requiring restricted access and careful data handling.
- False claims remain possible, so authorized staff must verify ownership, especially for valuable items. Some users may continue using informal channels; awareness and ease of use are important.

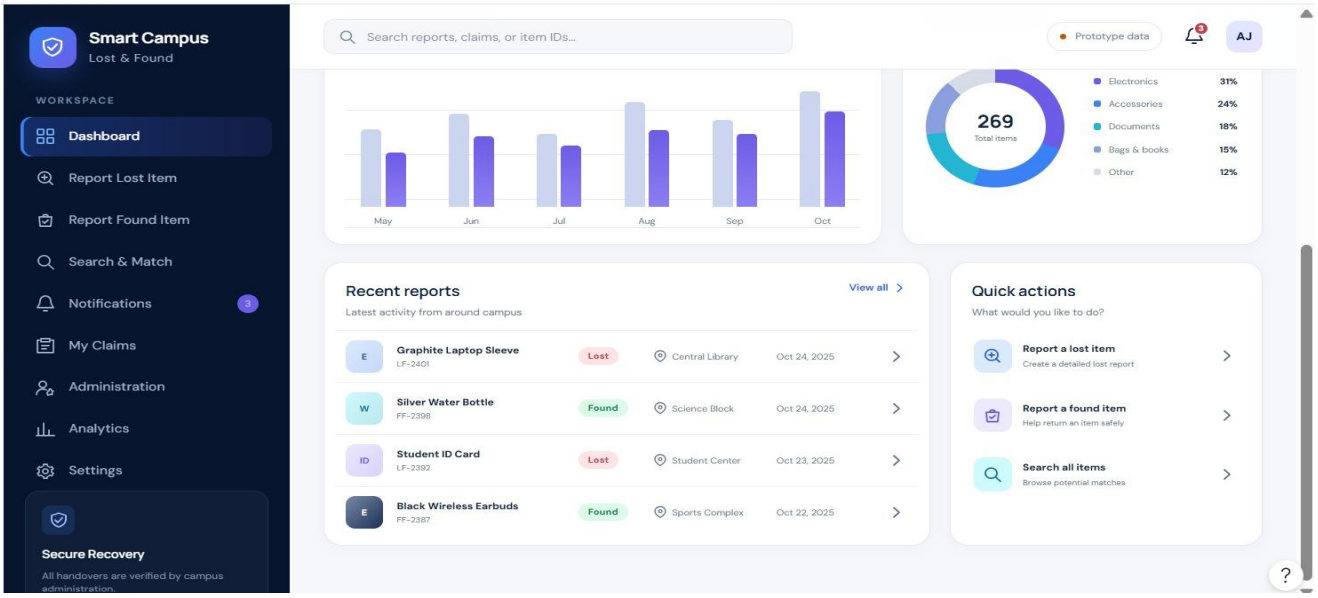
6.4 Future Enhancements

- AI-based image similarity to compare uploaded item photographs.
- Advanced matching using descriptions, categories, locations, timestamps and image features.
- Android/iOS mobile application and email or SMS notifications. QR codes at campus collection points for easier registration and tracking. Digital claim verification and audit logs.
- Multi-campus support for use across several institutions.

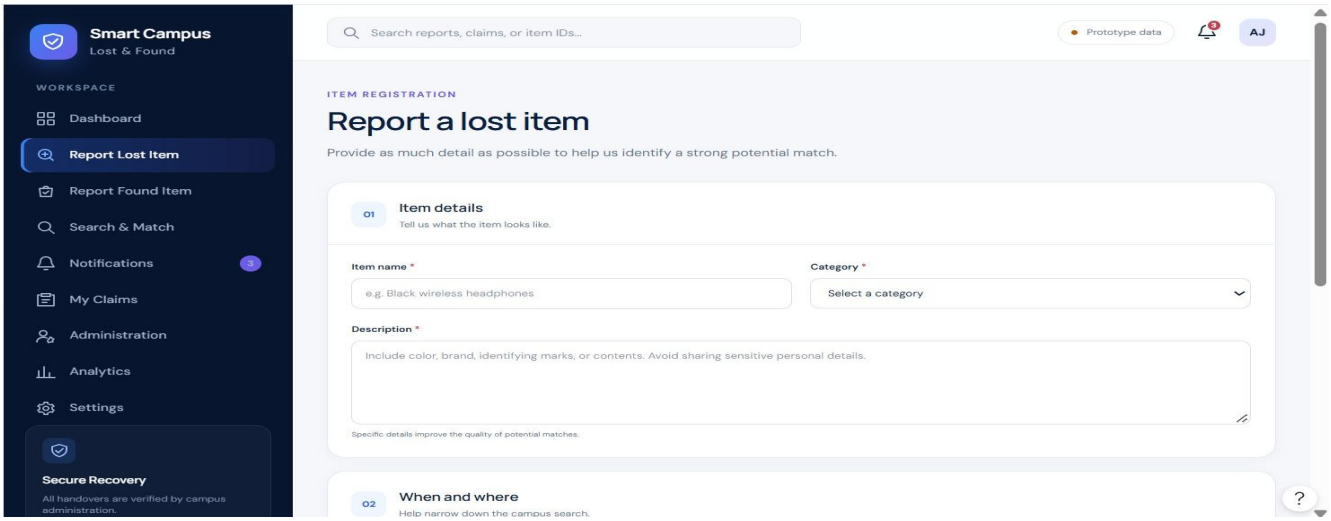
Conclusion

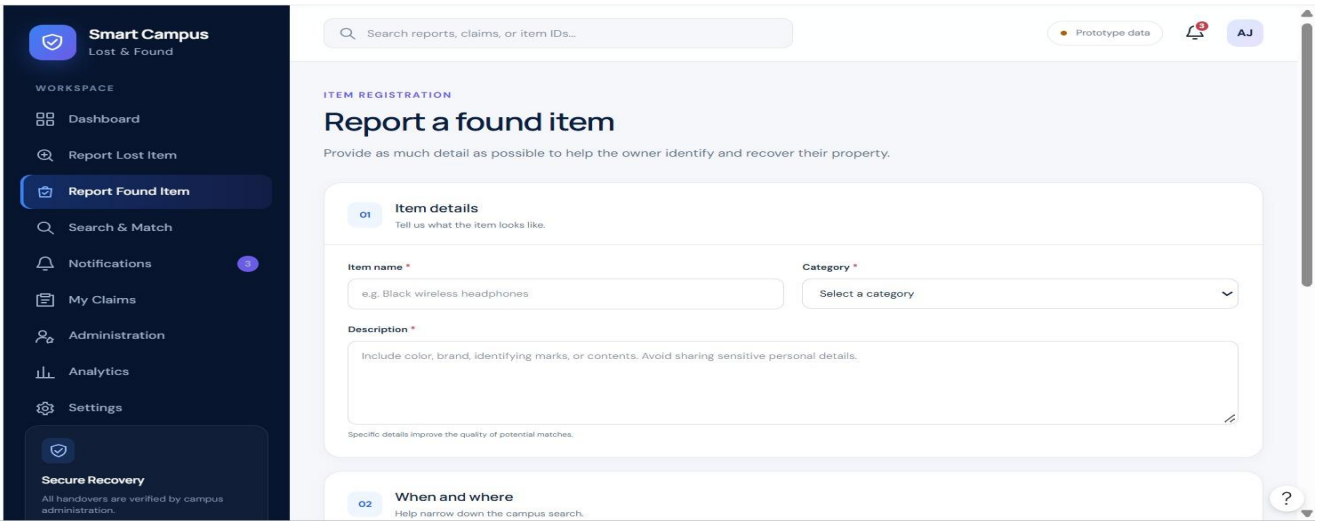
The Smart Campus Lost-and-Found Management System offers a centralized software solution to the limitations of manual campus lost-and-found processes. Structured reporting, searching, potential-match identification, notifications, ownership verification and recovery histories can make the process more organized, efficient and secure. Agile supports incremental implementation, testing and feedback.

IMPLEMENTATION SCREENSHOT (USING FIGMA TOOL)

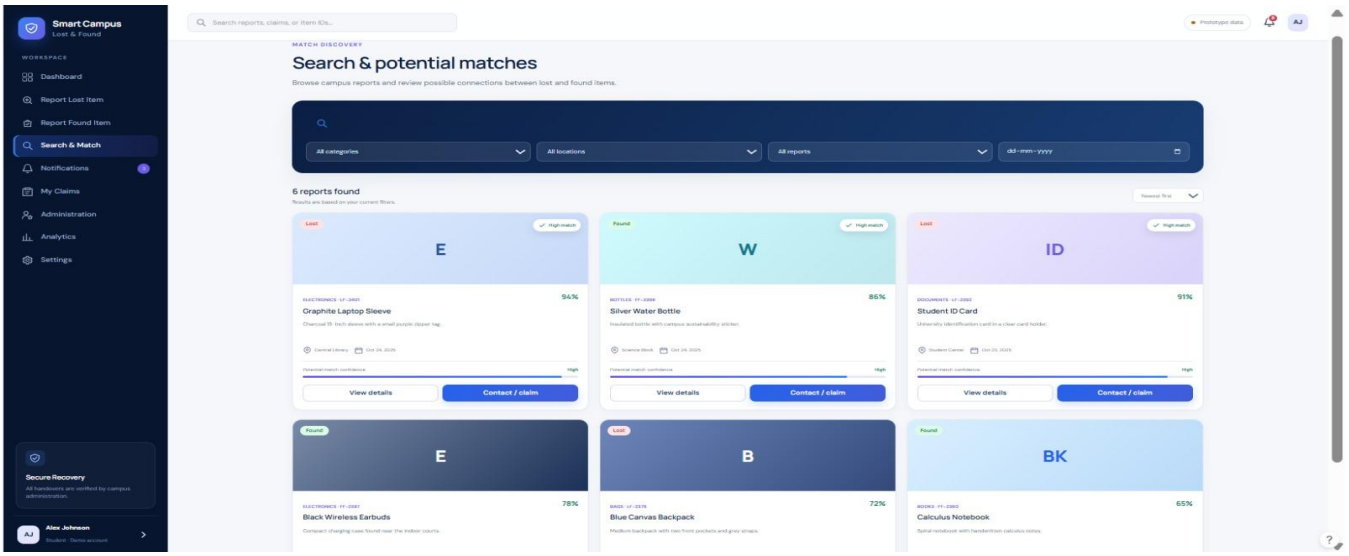


MODULE – 1 : Lost and Found Item Registration

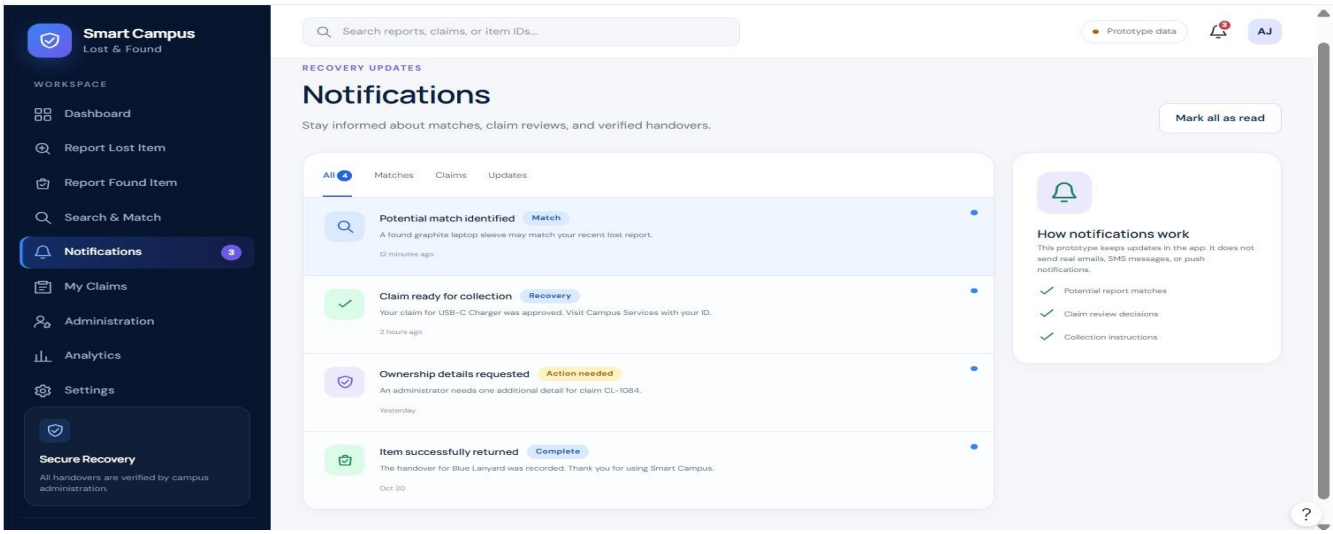


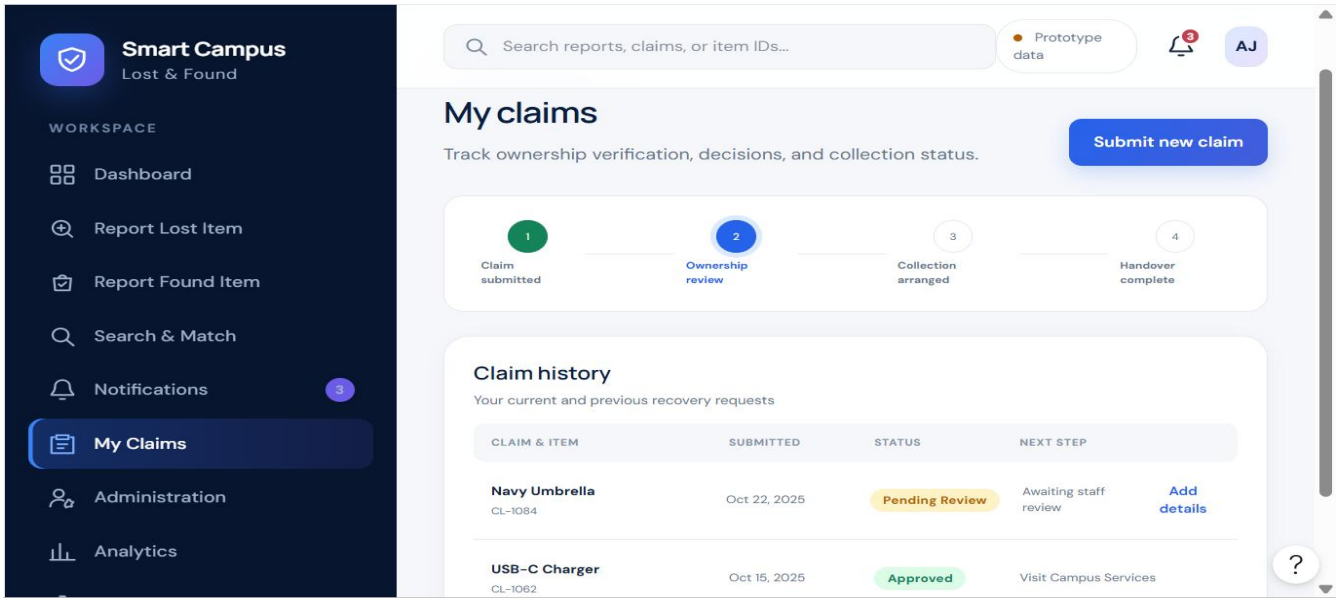


MODULE – 2 : Item Search and Potential-Match Identification



MODULE – 3 : Notifications and Secure Item Recovery





MODULE – 4 : Administration and Analytics

